TECHNICAL MODEL & MLOPS REPORT

PREDICTIVE MAINTENANCE

Reproducible model development, evaluation, explainability, and production-readiness design

Business question

Can routine machine operating conditions identify elevated failure risk early enough to support better maintenance decisions?

10,000	339	3.39%	7
observations	failures	failure prevalence	model configurations

Jupyter Notebook File: https://github.com/pateldev02/predictive-maintenance
Dataset File: https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset

TECHNICAL OVERVIEW

1. Analytical Design and Decision Objectives

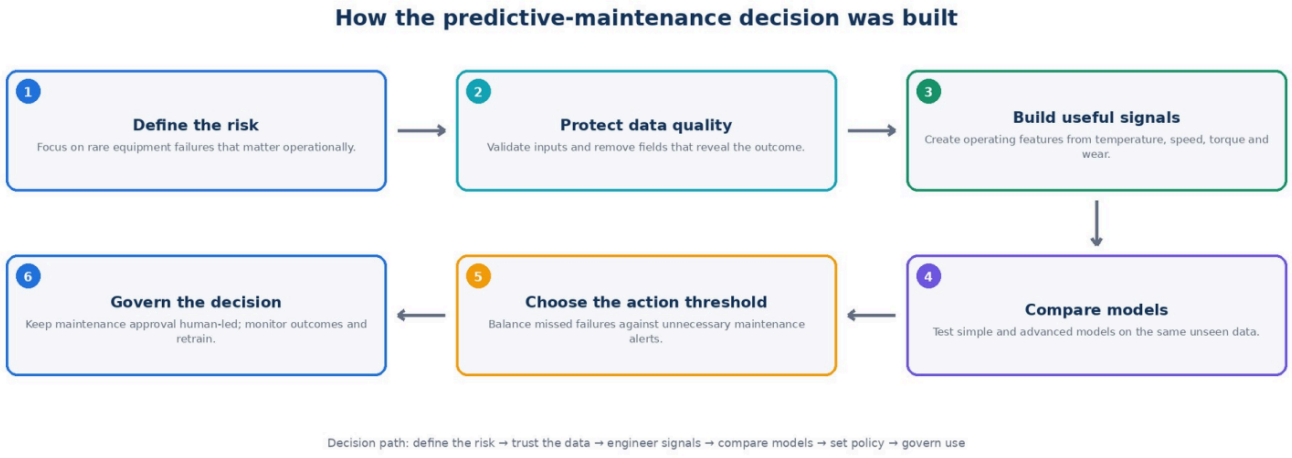
A reproducible account of how the machine-failure classifier was framed, built, challenged, selected, and prepared for controlled operational use.

Problem formulation

Binary classification target: Machine failure (1) versus normal operation (0). Predictors were limited to information that could plausibly be available before the failure outcome. The objective was not maximum accuracy; it was useful discrimination of a rare positive class with an explicit precision-recall trade-off.

Core technical decisions

Decision	Implementation	Reason
Leakage prevention	Exclude TWF/HDF/PWF/OSF/RNF and identifiers	Avoid outcome information and non-operational IDs
Split	80/20 stratified train/test; random_state=42	Preserve 3.39% failure prevalence
Preprocessing	One-hot encode Type; standardize numeric features in pipeline	Consistent train/test transformations and no preprocessing leakage
Metrics	Precision, recall, F1, ROC-AUC, PR-AUC	Rare-event evaluation
Candidates	Dummy, Logistic Regression, Random Forest, XGBoost + imbalance variants	Compare linear, bagging and boosting approaches
Selection	Model + threshold evaluated separately	Separate ranking quality from operational decision policy



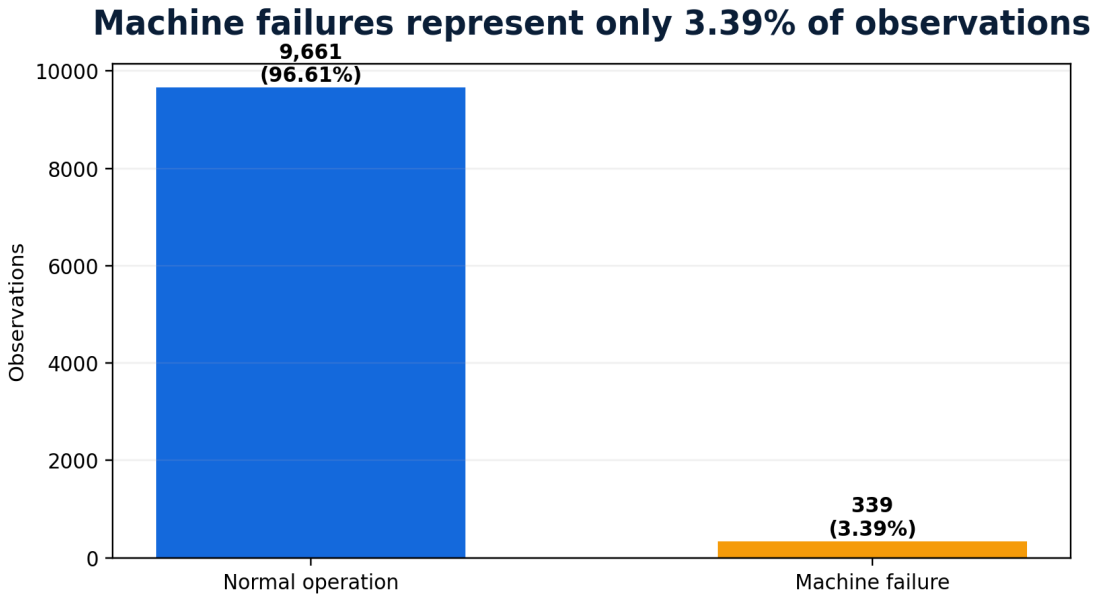
DATASET

2. Data Structure and Quality Assessment

The source contained 10,000 observations and 14 variables spanning identifiers, machine type, operating measurements, target, and failure-mode flags.

Variable group	Variables	Treatment
Identifiers	UDI, Product ID	Excluded from model
Categorical	Type	One-hot encoded
Raw numeric	Air temp, process temp, rotational speed, torque, tool wear	Retained
Target	Machine failure	Binary target
Failure flags	TWF, HDF, PWF, OSF, RNF	Excluded as potential leakage

10,000	14	339	3.39%
rows	source columns	failure rows	positive class



Quality checks

- No missing values were identified.
- No completely duplicated rows were identified.
- Machine type distribution: L=6,000; M=2,997; H=1,003.

- Failure-mode counts were reviewed to understand label structure before excluding them from predictors.

EDA

3. Exploratory Analysis and Operating Hypotheses

EDA was used to generate testable feature hypotheses, not to prove causation.

Feature	Normal mean/median	Failure mean/median	Technical observation
Air temperature	299.97 K mean	300.89 K mean	Small aggregate shift
Process temperature	310.00 K mean	310.29 K mean	Small aggregate shift
Rotational speed	1,507 rpm median	1,365 rpm median	Large median shift / nonlinear behavior likely
Torque	39.63 Nm mean	50.17 Nm mean	Strong load separation
Tool wear	107 min median	165 min median	Strong wear separation

Machine type failure rate

Type	Observations	Failures	Failure rate
H	1,003	21	2.09%
L	6,000	235	3.92%
M	2,997	83	2.77%

Interpretation discipline

Observed associations informed modeling and feature engineering, but were not treated as root-cause findings. Nonlinear interactions were expected, motivating tree-based models.

FEATURE ENGINEERING

4. Physics-Informed Feature Engineering

Three derived variables were added to represent thermal state, mechanical work, and wear under load.

Feature	Definition	Rationale

Temperature difference	Process temperature - air temperature	Represents thermal gap / heat-dissipation context
Angular velocity	RPM × 2π / 60	Converts rotational speed to rad/s
Mechanical power	Torque × angular velocity	Combines torque and rotational speed into work rate
Tool-wear load	Tool wear × torque	Interaction of accumulated wear and load

The final modeling feature set included Type, five raw operating variables, temperature difference, mechanical power and tool-wear load. The usefulness of engineered features was assessed within model performance rather than assumed a priori.

Pipeline design

A Scikit-learn ColumnTransformer handled categorical and numerical preprocessing, and each estimator was placed in a Pipeline. This ensured preprocessing was fit on training data only and consistently reused for inference.

BASELINE

5. Baseline and Linear Model Evaluation

The baseline established why rare-event metrics matter and how class weighting changes the decision surface.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Naive Baseline	0.966	0.000	0.000	0.000	0.500	0.034
Logistic Regression	0.968	0.571	0.176	0.270	0.926	0.464
Balanced Logistic Regression	0.858	0.179	0.882	0.298	0.939	0.430

Dataset Size	Training Set	Test Set
10,000	8,000 (80%)	2,000 (20%)

Interpretation

- Dummy majority classifier: 96.6% accuracy but zero true failures detected.
- Standard Logistic Regression: reasonable precision but only 17.6% recall, indicating a conservative boundary.
- Balanced Logistic Regression: recall increased to 88.2%, but precision dropped to 17.9%, creating a very high false-alert burden.

Technical lesson

Class weighting can move the decision boundary substantially. A higher recall value is not automatically better if precision collapses and the operational false-positive cost becomes unacceptable.

TREE MODELS

6. Random Forest Evaluation

Bagged decision trees were evaluated to capture nonlinear interactions without imposing a linear decision boundary.

Configuration	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	.992	.965	.809	.880	.973	.891
Balanced Random Forest	.990	.961	.721	.824	.971	.873

Decision

The standard Random Forest became the leading candidate. Class weighting reduced recall and PR-AUC, so the balanced variant was rejected. This result demonstrated that imbalance handling needed to be empirical rather than formulaic.

Random Forest strengths for this problem

- Captures nonlinear thresholds and feature interactions.
- Robust to mixed feature scales after pipeline preprocessing.
- Provides probability estimates for threshold optimization.
- Offers global importance measures that can be cross-checked with permutation importance.

BOOSTING CHALLENGER

7. XGBoost Evaluation and Hyperparameter Rationale

Gradient boosting was used as the primary challenger because it is strong on structured tabular data and optimizes sequential residual errors.

Parameter	Value	Role
n_estimators	300	Number of boosting trees
learning_rate	0.05	Contribution of each new tree
max_depth	4	Limits tree complexity
subsample	0.8	Row sampling to reduce overfit

colsample_bytree	0.8	Feature sampling per tree
objective	binary:logistic	Binary probability model
eval_metric	logloss	Probability-quality loss during training
random_state	42	Reproducibility

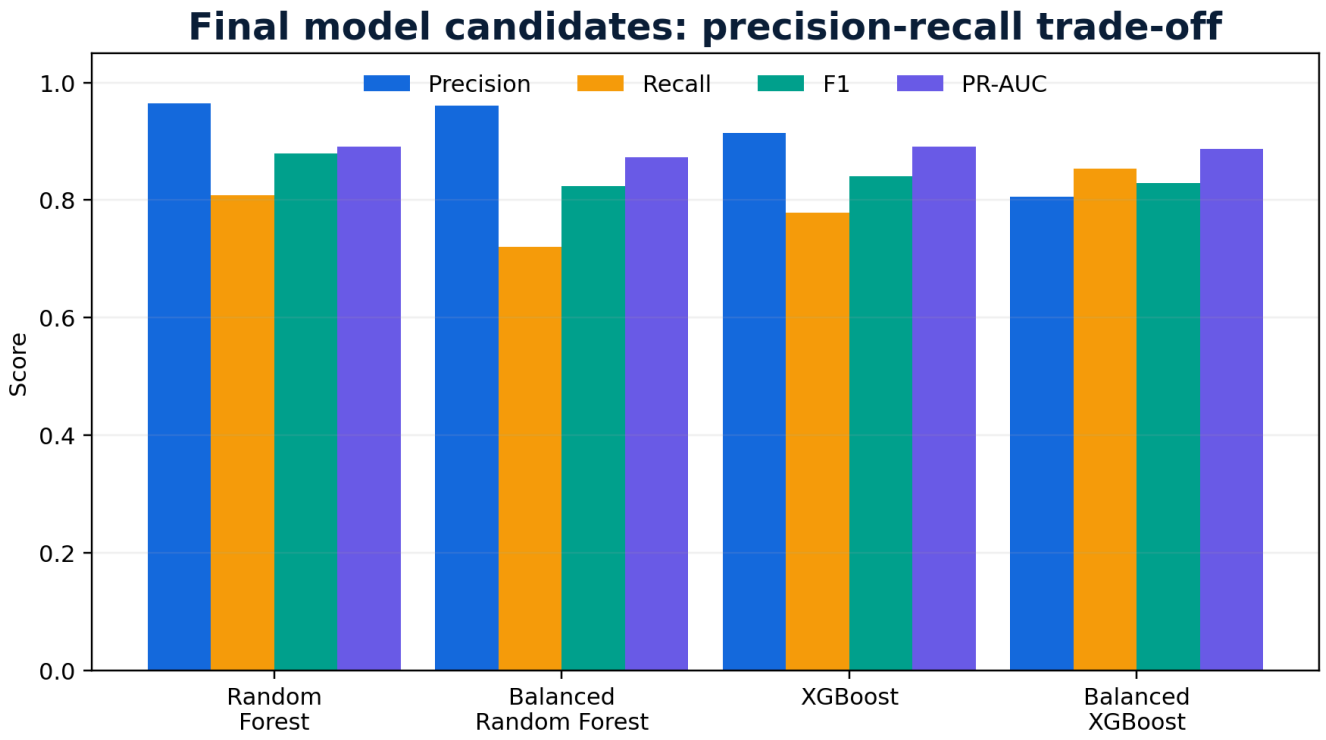
Configuration	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost	.914	.779	.841	.986	.891
Balanced XGBoost	.806	.853	.829	.983	.887

XGBoost achieved the highest ROC-AUC (0.986) and tied Random Forest on PR-AUC (0.891), indicating excellent ranking ability. However, Random Forest delivered a better default precision-recall balance and higher F1.

MODEL COMPARISON

8. Model Comparison and Selection

Model ranking quality and classification performance at a specific threshold were evaluated separately.



Selection criterion	Random Forest	XGBoost	Technical conclusion
Default F1	.880	.841	Random Forest

Default precision	.965	.914	Random Forest
Default recall	.809	.779	Random Forest
ROC-AUC	.973	.986	XGBoost
PR-AUC	.891	.891	Tie
85%+ recall precision	.787	.663	Random Forest

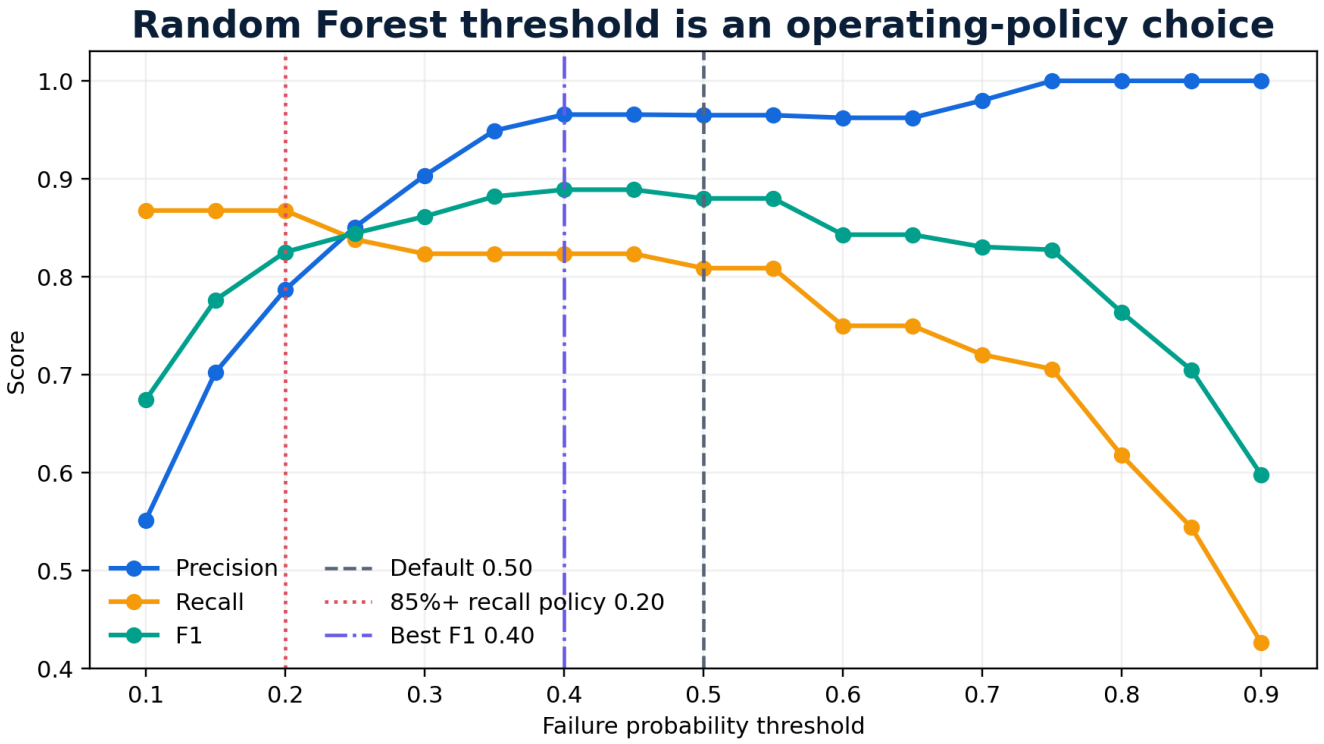
Selection rationale

XGBoost ranked observations extremely well, but Random Forest produced the better operational classifier at both the default threshold and the high-recall business constraint.

THRESHOLD ANALYSIS

9. Probability Threshold Optimization

Threshold optimization separated model training from the business decision rule.



Threshold	Precision	Recall	F1	Use case
0.20	78.7%	86.8%	82.5%	Failure-detection-first policy
0.40	96.6%	82.4%	88.9%	Best F1 in evaluated grid

0.50	96.5%	80.9%	88.0%	Default high-confidence policy

For the business-oriented requirement of at least 85% recall, Random Forest achieved 86.8% recall at threshold 0.20 with 78.7% precision. XGBoost reached 86.8% recall at threshold 0.10 but precision fell to 66.3%.

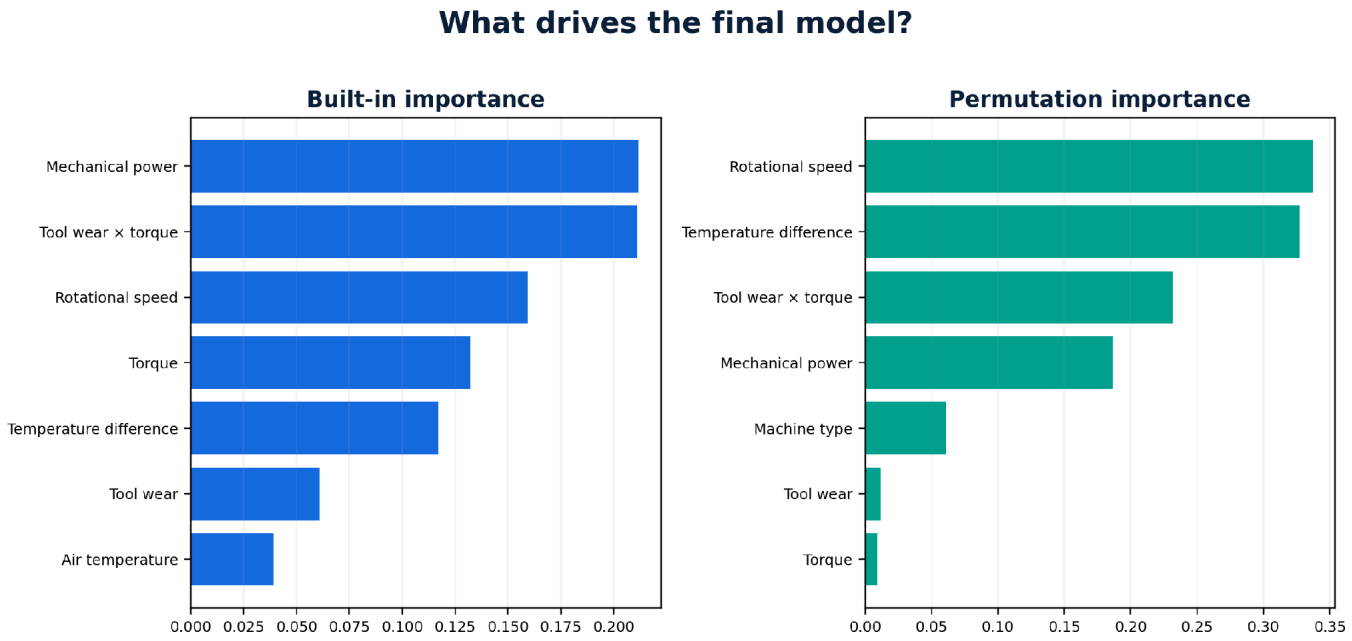
Validation caution

The threshold grid was evaluated on the held-out test set. In a production-quality workflow, threshold selection should be tuned on validation data or cross-validation and the final test set reserved for one-time unbiased evaluation.

EXPLAINABILITY

10. Global Model Interpretation

Two complementary importance methods were used to understand which predictors the final model relied upon.



Built-in tree importance

Mechanical power and tool-wear load were the largest built-in importance contributors, followed by rotational speed, torque and temperature difference.

Permutation importance

Randomly shuffling individual features produced the largest F1 degradation for rotational speed, temperature difference and tool-wear load. Process temperature had approximately zero/slightly negative permutation importance in this test set, suggesting limited incremental value after accounting for correlated variables.

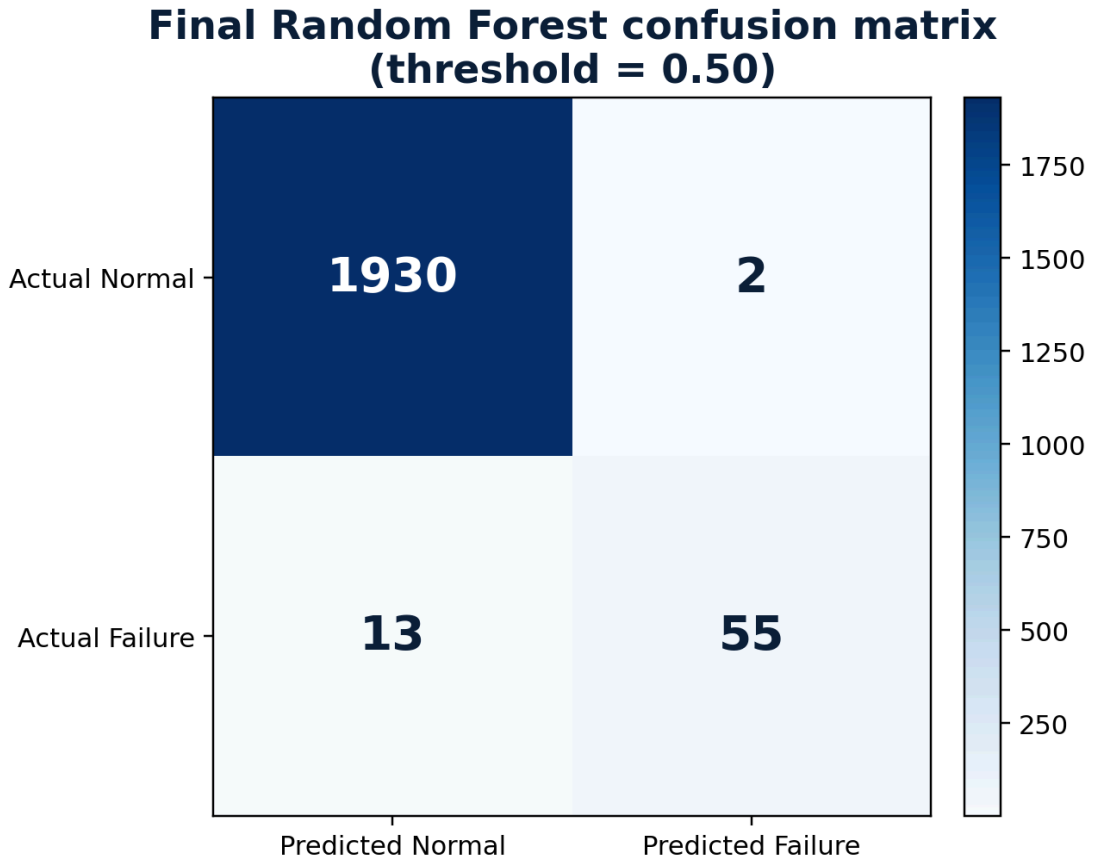
Correlation caveat

Importance can be split across correlated raw and engineered variables. Global importance describes predictive dependence, not causality or a safe operating limit.

FINAL EVALUATION

11. Hold-out Test Performance and Error Profile

The final Random Forest pipeline was evaluated at threshold 0.50.



0.992	0.965	0.809	0.880	0.973	0.891
accuracy	precision	recall	F1	ROC-AUC	PR-AUC

Confusion matrix counts: TN=1,930; FP=2; FN=13; TP=55. The default operating point is precision-oriented: only two false alerts occurred, while 13 of 68 failures were missed.

Alternative operating point

At threshold 0.20, recall increased to 86.8% and predicted failures increased from 57 to 75. This higher-recall configuration should only be selected after quantifying the maintenance burden and economic value of the additional captured failures.

MODEL ARTIFACT

12. Reproducibility and Inference Design

The final preprocessing-plus-model pipeline was serialized as a single reusable artifact.

Saved artifact

The notebook saved the Random Forest pipeline to predictive_maintenance_random_forest.pkl using joblib. Reloading the artifact reproduced approximately 99.25% accuracy on the same held-out test data.

Why the pipeline artifact matters

- One-hot encoding and numeric scaling travel with the estimator.
- New observations receive the exact transformations used during model training.
- The artifact can be versioned and tested as a single inference unit.

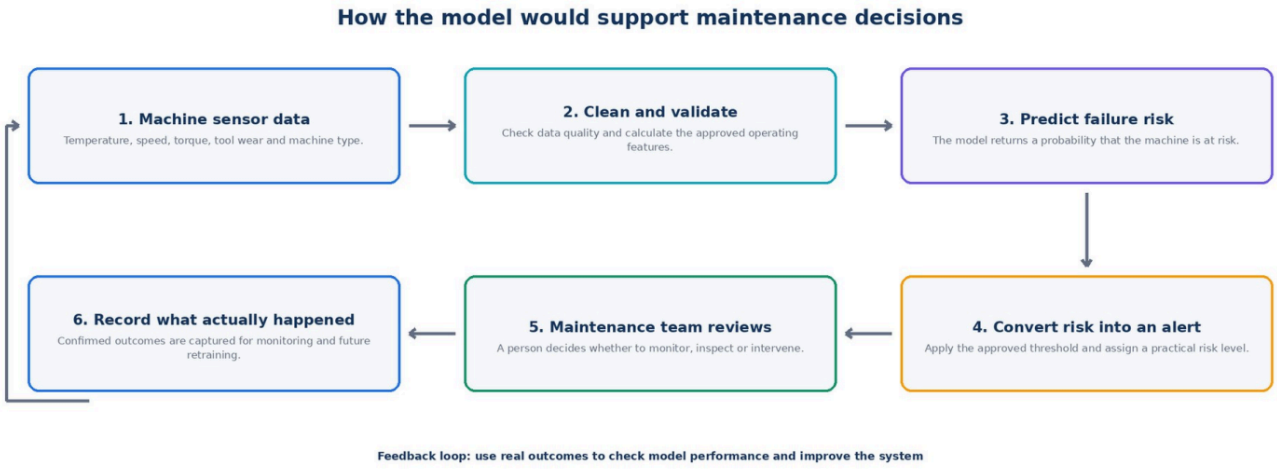
Production metadata to add

Metadata	Purpose
Model version / Git commit	Reproducibility and rollback
Training dataset version	Lineage
Feature schema version	Input compatibility
Threshold version	Decision-policy traceability
Training date + metrics	Performance history
Approval state / owner	Governance

PRODUCTION ARCHITECTURE

13. Recommended MLOps extension

The notebook is sufficient for proof of concept; production use requires data, software, model, and governance controls.



Layer	Recommended capability
Data ingestion	Historian / PLC / sensor ingestion with timestamp and asset identity controls
Validation	Schema, range, missingness, unit and sensor-quality tests
Feature pipeline	Versioned feature transformations with train/inference parity
Model serving	Batch or API scoring with versioned model + threshold
Decision integration	CMMS/work-order or monitoring dashboard with human approval
Monitoring	Input drift, prediction drift, alert rate, outcome metrics, latency
Retraining	Governed trigger, validation gates and rollback

VALIDATION AND MONITORING

14. Enterprise acceptance criteria

A technically strong model is not production-ready until performance remains stable under real operating conditions.

Recommended validation tests

- Time-based holdout to test future generalization.
- Asset-level or site-level holdout to test transferability.
- Cross-validation within training data for hyperparameter and threshold tuning.
- Calibration analysis to verify whether predicted probabilities reflect observed risk.
- Stress tests for missing sensors, out-of-range values, new machine types and distribution shifts.
- Subgroup metrics by asset type, line, site, shift and operating regime.

Monitoring metrics

Category	Examples
Data quality	Missingness, range violations, stale sensors, schema drift

Drift	PSI/KS or distribution change by critical feature
Prediction behavior	Alert rate, score distribution, threshold crossings
Outcome performance	Precision, recall, F1, PR-AUC when labels mature
Operational	Inspection volume, confirmed faults, avoided downtime, latency
Governance	Model version, threshold changes, retraining approvals, rollback events

LIMITATIONS

15. Technical limitations and future improvements

The proof of concept is strong enough to justify further validation, but not to support unrestricted deployment.

- Simulated/public dataset: real plant data may contain different noise, failure mechanisms, maintenance practices and sensor quality.
- Only 339 failures: rare-event estimates can be sensitive to sample composition.
- Snapshot observations: no true machine-level degradation sequence or time-to-failure target.
- Same-source train/test split: does not demonstrate generalization across sites, assets or time.
- Threshold optimization touched the held-out test set: future work should use validation data/cross-validation for threshold selection.
- No probability calibration study was performed.
- No hyperparameter search or nested validation was performed; XGBoost parameters were reasonable starting values, not proven optima.
- Feature importance does not establish root cause.

High-value future improvements

- Collect longitudinal telemetry and maintenance-event history to model time-to-failure and degradation trends.
- Add rolling statistics, rate-of-change and vibration/current/pressure features when real time-series data is available.
- Use cross-validated hyperparameter optimization and probability calibration.
- Evaluate cost-sensitive objective functions using real downtime and inspection costs.
- Add SHAP or local explanations for individual high-risk predictions.
- Implement experiment tracking/model registry, drift monitoring, CI/CD pipeline and testing, and controlled retraining.

Final technical conclusion

Random Forest was selected because it delivered the strongest operational precision-recall balance, not because it dominated every metric. XGBoost had the best ROC-AUC, but Random Forest converted ranking quality into a better decision policy under both default and 85%+ recall scenarios.